

TASK 1

Write a program to display the functionality of method overloading

```
public class method_overloading {  
  
    void add(int a , int b) {  
        System.out.println(" value: " + (a+ b));  
    }  
  
    void add (int a, int b, int c) {  
        System.out.println("Sum: " + (a + b + c));  
    }  
  
    public static void main(String[] args) {  
  
        method_overloading obj = new method_overloading();  
  
        obj.add(10,20);  
        obj.add(10, 20, 45);  
  
    }  
}
```

Task 2:

Write a brief history of Java

- Java is a high-level, object-oriented programming language

It is mainly known for its simplicity, security, robustness

```
import java.security.spec.RSAOtherPrimeInfo;  
import java.util.LinkedList;  
  
public class Day3_linkedlist {  
    public static void main(String[] args) {  
        LinkedList<String> Name = new LinkedList<>();  
        Name.add("MONIKA");  
        Name.add(" LIKITHA ");  
        Name.add(" RUCHI");  
        Name.add(" HARSHA ");  
        Name.add(" PRAGNA");  
        Name.add("THANU ");  
        System.out.println(Name);  
  
        Name.remove( 3 );  
        System.out.println(Name);  
  
        Name.add("ROHAN");  
        System.out.println(Name);  
  
        Name.addFirst("GOPIKA");  
        System.out.println(Name);  
  
        Name.addLast("BHARAGAV");  
        System.out.println(Name);  
    }  
}
```

```
}  
}
```

Output

[MONIKA, LIKITHA , RUCHI, HARSHA , PRAGNA, THANU]

[MONIKA, LIKITHA , RUCHI, PRAGNA, THANU]

[MONIKA, LIKITHA , RUCHI, PRAGNA, THANU , ROHAN]

[GOPIKA, MONIKA, LIKITHA , RUCHI, PRAGNA, THANU , ROHAN]

[GOPIKA, MONIKA, LIKITHA , RUCHI, PRAGNA, THANU , ROHAN, BHARAGAV]

[GOPIKA, MONIKA, LIKITHA , RUCHI, PRAGNA, THANU , ROHAN, BHARAGAV]

Task 3

Swapping number

```
public class Day3_swapnumber {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 40;  
        int temp;  
        System.out.println( "value of a and b : " + a + " " + b );  
        temp = a;  
        a = b;  
        b = temp;  
        System.out.println(" swaped value of a nad b :"+a + " " +b);  
    }  
}
```

Task 4

Customised LinkedList for creating the node

```
class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

```

public class Day3_custLinkedList {

    Node head;

    void add(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            return;
        }

        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    void remove(int data) {
        if (head == null) {
            return;
        }

        if (head.data == data) {
            head = head.next;
            return;
        }

        Node temp = head;

        while (temp.next != null && temp.next.data != data) {
            temp = temp.next;
        }

        if (temp.next != null) {
            temp.next = temp.next.next;
        }
    }

    void display() {
        Node temp = head;

        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }

        System.out.println("null");
    }

    public static void main(String[] args) {

        Day3_custLinkedList list = new Day3_custLinkedList();

        list.add(10);
        list.add(20);
        list.add(30);
    }
}

```

```

        System.out.println("After Adding:");
        list.display();

        list.remove(20);

        System.out.println("After Removing 20:");
        list.display();
    }
}

```

Task 5

Wap to add and display elements in a built in stack

Import java.util.Deque;

Import java.util.ArrayDeque;

No stack — legacy code

```

import java.util.Deque;
import java.util.ArrayDeque;
public class Day4_Stack {
    public static void main(String[] args) {
        Deque<Integer> stack = new ArrayDeque<>();

        stack.push(100);
        stack.push(200);
        stack.push(300);
        stack.push(400);

        System.out.println("Stack Elements:");
        for (Integer element : stack) {
            System.out.println(element);
        }

        stack.pop();
        System.out.println("after popping ");
        for (Integer element1 : stack){
            System.out.println(element1);
        }
    }
}

```

Task 6

Wap to create a builtin Queue and display the elements

Import java.util.LinkedList

Import java.util.Queue;

```
import java.util.LinkedList;
import java.util.Queue;

public class Day4_queue {
    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();
        queue.add(100);
        queue.add(200);
        queue.add(300);
        queue.add(400);
        queue.add(500);
        System.out.println("Queue Elements: " + queue);

        queue.remove();
        System.out.println(" Queue elements after removing" + queue);
    }
}
```

Task 7:

Wap to demonstrate Method overriding

```
class Bank {
    void getRateOfInterest() {
        System.out.println("Interest Rate: 5%");
    }
}

class SBI extends Bank {
    void getRateOfInterest() {
        System.out.println("Interest Rate: 7%");
    }
}

public class Method_overriding{
    public static void main(String[] args) {

        Bank b = new SBI();
        b.getRateOfInterest();
    }
}
```

Task 8:

What are the advantages of Polymorphism.. Brief it

- Polymorphism is an OOPs concept that allows a single method or object to take many forms.
- It is achieved through method overloading and method overloading.
- Advantages:
 - ☐ Increases code reusability.
 - ☐ Provides flexibility and extensibility.
 - ☐ Reduces code complexity.
 - ☐ Improves maintainability.
 - ☐ Supports runtime polymorphism and dynamic method dispatch.

Task 9:

Wap to show the encapsulation feature in java

Use getters and setters

```
class Student {
    private String name;
    private int age;

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

public class Day4_encapsulation {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Monika");
        s.setAge(22);
        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
    }
}
```

Task 10:

What are the advantages and disadvantages of encapsulation

- Encapsulation provides data security by hiding data and allowing access through getter and setter methods. It improves code maintainability, flexibility, and reusability while preventing unauthorized access to data.
- Disadvantage - it can increase code length because additional getter and setter methods are required and may add slight complexity to the program.

Task 11:

Wap to use the concept of type casting:

```
public class Day4_typecasting {  
    public static void main(String[] args) {  
        int number = 10;  
        double d = number;  
  
        System.out.println("Before typecasting : " + number);  
        System.out.println("After typecasting : " + d);  
    }  
}
```

Wrapperclass

```
public class Day4_Wrapperclass {  
    public static void main(String[] args) {  
  
        double marks = 85.5;  
  
        Double obj = marks;  
  
        System.out.println("Primitive Value: " + marks);  
        System.out.println("Wrapper Object: " + obj);  
    }  
}
```

Task 12:

What is auto boxing and auto unboxing

- Autoboxing is the automatic conversion of a primitive data type wrapper class object.
- Unboxing is the automatic conversion of a wrapper class object primitive data type.

Multi_threading

```
- class RunnableDemo implements Runnable {  
    private Thread t;  
    private String threadName;
```

```

RunnableDemo( String name){
    threadName = name;
    System.out.println("Creating " + threadName );
}
public void run() {
    System.out.println("Running " + threadName );
    try {
        for(int i = 4; i > 0; i--) {
            System.out.println("Thread: " + threadName + ", " + i);
            // Let the thread sleep for a while.
            Thread.sleep(50);
        }
    } catch (InterruptedException e) {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}

public void start ()
{
    System.out.println("Starting " + threadName );
    if (t == null)
    {
        t = new Thread (this, threadName);
        t.start ();
    }
}

public class TESTTHREAD {
    public static void main(String args[]) {
        RunnableDemo R1 = new RunnableDemo( "Thread-1");
        R1.start();
        RunnableDemo R2 = new RunnableDemo( "Thread-2");
        R2.start();
    }
}

```

Home tasks:

1. Write a program to create a doubly linked list

```

import java.util.Scanner;

class Node1 {
    int data;
    Node1 prev, next;

    Node1(int data) {
        this.data = data;
        prev = null;
        next = null;
    }
}

public class DoublyLinkesList {
    Node1 head = null;

    void insert(int data) {

```



```

Node1 newNode = new Node1(data);

if (head == null) {
    head = newNode;
    return;
}

Node1 temp = head;
while (temp.next != null) {
    temp = temp.next;
}

temp.next = newNode;
newNode.prev = temp;
}

void display() {
    Node1 temp = head;

    System.out.print("Doubly Linked List: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    DoublyLinkesList list = new DoublyLinkesList();

    System.out.print("Enter number of nodes: ");
    int n = sc.nextInt();

    for (int i = 1; i <= n; i++) {
        System.out.print("Enter data for node " + i + ": ");
        int data = sc.nextInt();
        list.insert(data);
    }

    list.display();
    sc.close();
}
}

```

2. Write a program to create a circular linked list

```

import java.util.Scanner;
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

```

```

public class CircularLinkedList {
    Node head = null;
    Node tail = null;

    void insert(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            tail = newNode;
            tail.next = head;
        } else {
            tail.next = newNode;
            tail = newNode;
            tail.next = head;
        }
    }

    void display() {
        if (head == null) {
            System.out.println("List is empty");
            return;
        }

        Node temp = head;
        System.out.print("Circular Linked List: ");
        do {
            System.out.print(temp.data + " ");
            temp = temp.next;
        } while (temp != head);

        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CircularLinkedList list = new CircularLinkedList();

        System.out.print("Enter the number of nodes: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("Enter data for node " + i + ": ");
            int data = sc.nextInt();
            list.insert(data);
        }

        list.display();
        sc.close();
    }
}

```